Computer Science and Information Systems 00(0):0000–0000 https://doi.org/10.2298/CSIS123456789X

Paper Type: Research Paper

Cardinality Coverage in Evolutionary Feature Selection: A Budget-Aware Study of Initialization and Search Operators

Klemen Kenda[1⋆]

Jožef Stefan Institute
Jamova cesta 39, 1000 Ljubljana, Slovenia
klemen.kenda@ijs.si

Abstract. Evolutionary feature-selection methods are often compared at small feature limits, where a score mixes whether a method returns any subset within the limit with how well its subsets predict. We separate them for three NSGA-II configurations (an opposition-based start, a sparse start, and our implementation of NSGAII-MIIP) on 46 datasets with 16 to 192,000 features and up to three classifiers, plus a four-dataset pilot and a confirmatory test on seven new datasets. At a limit of ten features, reach makes up 98 to 100 percent of the combined absolute reach and quality parts of the area-under-the-front difference between the two starts. At sizes both starts reach, mean test F1 differences lie within 0.02 under two classifiers and are inconclusive under the third. At benchmark budgets, the opposition-based start never reaches ten features beyond 512 features. The fits it needs to get there grow as about $d^{1.7}$ with the number of features d: a prediction fitted on the pilot and registered beforehand held within a factor of two on all seven new datasets, which have 128 to 1,024 features. Comparisons should therefore report reach, matched-size scores, and the budget.

Keywords: evolutionary feature selection, multi-objective optimization, initialization, cardinality coverage, evaluation budget.

1. Introduction

Feature selection chooses the features on which a prediction model is trained. A wrapper method judges each candidate subset of features by training the model with it and measuring how well the model predicts. When predictive quality and the number of selected features, called the subset size or *cardinality*, are treated as two separate objectives, the result is not a single subset but a set of trade-offs from which a user can choose [28,12]. Evolutionary multi-objective algorithms, such as the nondominated sorting genetic algorithm II (NSGA-II) [8], are a common way to search for such a set. A run returns a front of subsets in which each larger subset predicts better, on the data used by the search, than every smaller one. Small subsets often matter most in practice: a model with few features is easier to inspect and explain, each feature may be costly to measure or compute, and

⋆ Corresponding author

a deployed model needs only the features it uses. Fronts are therefore often compared at small feature limits, for example with the area under the front (AUF) introduced with the FASTENER method [15]. For each limit k up to a chosen maximum, such as ten features, AUF takes the best score available with at most k features and averages these scores over the limits.

A score at a feature limit answers two questions at once. The first is whether the method returns any subset within the limit. The second is how well the subsets it returns predict. AUF counts a limit as zero when the front holds no subset with at most that many features, so a zero there marks a missing subset, not a poor one. We say that a run *reaches* a limit if its returned front contains a subset within it, and we call the range of subset sizes on the front its *cardinality coverage*. A large AUF difference between two methods is easily read as better search among small subsets, when the method with the lower score may simply not have reached the small sizes. The two readings lead to different conclusions: the first concerns how well the search works among small subsets, the second whether it gets there at all within the available number of model fits.

How a search can fail to reach a small limit is visible in how it starts and how it changes subsets (Section 3). The opposition-based initialization [24], as used in the NSGAII-MIIP algorithm [17], selects each feature with probability one half and pairs each candidate with its opposite, so every candidate starts with about half of the features. On a dataset with a few hundred features, such a population almost never contains a subset with ten or fewer features, and the search must move down through crossover, mutation, and selection before a finite budget of model fits ends. Once subsets are small, the balanced mutation of NSGAII-MIIP [21,17] changes very few features, and many children repeat subsets evaluated before. A cache, which stores evaluation results so that a repeated subset needs no new fit, keeps these repeats out of the fit count, so they cost time without appearing in the budget.

The role of subset size at initialization is already recognized. Earlier work allocates initial candidates across size ranges, compares random and informed starts, and adds sparse candidates, that is, subsets with few selected features [9,16]. Miyandoab et al. [30] draw a subset size before selecting features and compare NSGA-II with and without this initialization, and later compare several population-based methods with it [31]. Cong et al. [6] compare sparse and dense starts at a fixed evaluation target and report that they differ in access to compact fronts in a gene-panel application. The unchanged expected size of the balanced mutation is stated in [21,17], and AUF comes from FASTENER [15]. These studies show that the start affects which subset sizes are available to the search, and controlled comparisons of initializations are themselves established. Our question is how the start affects the reading of a reported comparison. How much of a difference at a small feature limit is a difference in reach? When does the lack of reach occur, in terms of the number of features and the budget? What does a sparse start give up among large subsets? And what should be reported so that a reader can tell reach and quality apart?

To answer these questions, we compare three configurations of NSGA-II that share the genetic operators of NSGAII-MIIP: its opposition-based (OBL) start, in which each candidate begins with about half of the features; a sparse start, in which each candidate has 1 to 30 features; and our implementation of NSGAII-MIIP, which adds a mutual-information improvement phase to the OBL start. We run them on 46 datasets with 16 to 192,000 features, with up to three classifiers and five seeds, under a fixed budget of model

fits. A pilot on four of these datasets adds larger budgets, starts that cover all subset sizes, and a mutation that always changes a subset. A confirmatory test on seven further datasets measures how the number of fits the OBL start needs to reach small subsets grows with the number of features. We do not propose a new initialization, mutation, or evaluation measure; we use size-based initialization [30], balanced mutation [21,17], and AUF [15] as described in those works. The contributions are threefold:

1. **Evidence that a large AUF difference can come almost entirely from reach.** With a limit of ten features, the sparse start has a much higher AUF than the OBL start. A split of this difference into a reach part and a quality part is dominated by reach: the reach part makes up 98 to 100% of the combined absolute size of the two parts, and it comes mostly from limits that the OBL start never reaches. At sizes that both starts reach, which occur on at most 13 datasets per group, the mean difference in test F1 is small under two classifiers; under the random forest, the primary analysis is inconclusive. In exchange, the sparse start has a lower best test F1 at any size, so neither start covers the whole range of subset sizes.
2. **An account of when and why the search stays near its starting sizes.** We calculate the probability that an initial population contains a small subset and the expected number of features that balanced mutation changes, $2k/d$ for a subset of k out of d features. Within the benchmark budgets, the OBL search reaches ten features on every dataset with at most 99 features and on none with more than 512. In the pilot, ten times the budget is enough on four datasets with 176 to 720 features. On seven datasets not used before, the fits that the OBL search needs before it first holds a subset with at most ten features grow as about $d^{1.7}$ and stay within a factor of two of a prediction fitted on the pilot and registered before the runs. The improvement phase of our NSGAII-MIIP implementation raises reach only slightly. In sparse runs, almost all proposed subsets had been evaluated before, as the mutation analysis leads one to expect. The account is consistent with the observations; it does not establish a single cause.
3. **Recommendations for reporting comparisons at a feature limit.** From these results we derive a short list of quantities to report alongside AUF: reach and when it occurs, the score at sizes reached by both methods, the best score at any size, proposals, new model fits, elapsed time, and the runs that did not finish.

The results on 8 of the 46 datasets come from an experiment designed for this comparison. The results on the other 38 datasets reuse runs from a broader benchmark of feature-selection methods. We wrote the criteria of the analysis after the results and an exploratory analysis had been read, so we report it as a secondary analysis that describes the tested configurations, budgets, and classifiers, not as a confirmation of a general property of NSGA-II. A rule that sets aside the smallest test sets was added after this analysis and is reported only as an exploratory sensitivity analysis. The pilot was registered before its runs and uses new seeds, but its four datasets had been seen before, so it guides the interpretation and does not confirm it. Only the last experiment is confirmatory: the prediction of how many fits the OBL start needs was fitted on the pilot, registered with its criteria, and then tested on seven datasets on which no configuration had been run.

Section 2 reviews related work and states what this paper adds. Section 3 defines the measures and analyzes initialization and mutation, Section 4 describes the experiments,

Section 5 reports the results, and Sections 6 and 7 discuss and conclude. The supplementary material lists the datasets, gives the implementation details, and examines missing runs and the choice of classifier.

2. Related Work

Treating predictive quality and the number of selected features as separate objectives makes both the quality of the returned subsets and the range of their sizes relevant to a comparison [28,12]. The nondominated sorting genetic algorithm II (NSGA-II) provides the selection framework used in this study [8]. It ranks candidates by dominance and selects the next population from parents and offspring together, using crowding distance, a measure of spacing in the objective values, to favor less crowded parts of each front. These rules can be combined with different initializations, crossover operators, and mutation operators, so the behavior of a configuration also depends on those choices.

The number of selected features, also called subset cardinality, is an established consideration in population initialization. Previous work compares random and informed starts, allocates candidates across size ranges, and supplies sparse candidates, meaning subsets with few selected features [9,10,16]. These choices affect which subset sizes are available before evolutionary search begins. One explicit approach first samples a subset size uniformly within a given range and then selects that many features without replacement [30]. Miyandoab et al. compare NSGA-II with and without this initialization as well as their complete method, so a controlled comparison of initializations within one algorithm is itself prior art, and a later study extends the comparison to several population-based methods [31]. Bounded starts are also used in biomarker discovery: Cong et al. [6] attribute their start with at most 50 features to Cattelani and Fortino [4]. The bounded initialization used in our comparison follows the size sampling of Miyandoab et al. with a smaller range. Its role is to provide access to small subsets under the chosen budget; the sampling procedure itself is existing work.

Opposition-based initialization seeks diversity by pairing candidates with their opposites [24]. In the binary representation used by NSGAII-MIIP, the opposite selects every feature excluded by the original candidate and excludes every feature it selected [17]. With independent random inclusion at probability one half, both candidates have an expected size of half the available features, so complementary pairs neither spread the population across subset sizes nor guarantee small subsets.

Maintaining small subsets during search also depends on crossover, mutation, and the rule used to retain candidates. Methods that combine size-based initialization with adaptive crossover address both the initial population and the sizes of its descendants [29,27]. FASTENER, an evolutionary wrapper developed for land-cover classification from Earth observation data, uses feature information to guide crossover toward smaller subsets and keeps the search population separate from the stored Pareto front [15]. SparseEA and its feature-selection extensions combine initialization with feature scoring and search operators designed for sparse solutions [23,5]. Selection can also preserve different subset sizes by retaining candidates from separate size groups [20]. Together, these approaches show why an experiment that changes only the initial population differs from one that also changes how candidates are generated and retained.

The expected change in subset size and the amount of change produced by mutation are separate properties. Controlled mutation can balance the expected numbers of removed and added features [21,17], which keeps the expected subset size unchanged but leaves open how often any feature changes. Under the mutation rate examined here, that frequency depends on the current subset size relative to the total number of features.

An informed improvement phase provides another way to change the subsets produced by genetic operators. NSGAII-MIIP combines NSGA-II with an improvement phase guided by mutual information (MIIP) [17]. Mutual information measures how much knowing one variable reduces uncertainty about another; the phase uses it to add, remove, and exchange features by their relevance to the class and their redundancy with selected features, and its candidates are ranked together with the others. Including our implementation of the complete method separates the change in initialization from the additional search of the improvement phase.

A finite budget makes access to small subsets an empirical question even when the initialization can generate every subset size, because sampling from a broad range does not guarantee that a finite population contains a candidate within a particular small-size limit. Existing work addresses this through size sampling and later insertion of new candidates [30] and compares sparse and dense starts under a fixed evaluation target [6], so compact-front access is already a subject of experimental study.

The evaluation measure determines how differences in subset-size coverage appear in the reported result. FASTENER introduced the area under the front (AUF), which averages the best predictive score available with at most a given number of features over a range of feature-count limits [15]. Hypervolume, the area between the front and a chosen reference point, depends on the extent of the front, the objective values, and the reference point [1]. We use the AUF definition from FASTENER.

Repeated candidates connect search behavior to computational cost. Duplicate handling has been used to address lost diversity in evolutionary feature selection [13,26], while caching stores evaluation results so that a repeated subset needs no new fit [18]. Removing duplicates changes the candidates considered by the search, whereas reusing a stored result only saves a fit, so a run can generate many candidates while consuming few new fits. Reporting proposals, new fits, and elapsed time together follows the distinction between evaluation budgets and computational time in optimization benchmarking [3].

These studies establish the importance of initialization, subset-size control, front evaluation, and repeated candidates; our focus is how these factors affect the interpretation of a comparison within a specified configuration.

2.1. Our Contributions in Relation to Existing Work

Table 1 lists the contributions of this paper in order of importance, with the prior work each builds on; the results behind them are summarized in Section 1.

Two studies are closest to ours. Miyandoab et al. [30] described the size sampling that our sparse start uses, over all sizes and with later insertion of new candidates, and compared it with the default start within NSGA-II by hypervolume. We use it over 1 to 30 features, without insertion, as an experimental condition and not as a method of our own. Cong et al. [6] compare sparse and dense starts at a fixed evaluation target and report access to compact fronts in one gene-panel application. What we add is not the comparison of starts itself but its reading: the split of an AUF difference into reach and quality

Table 1. Contributions of this paper, in order of importance, and the prior work each builds on

Contribution	Builds on
Reach, not quality, explains the AUF gap at a limit of ten features	Initialization-only comparison in NSGA-II [30]; sparse and dense starts in one application [6]; AUF [15]
When reach fails: the initial population, the benchmark boundary, and fits growing as about $d^{1.7}$ on unseen datasets	Opposition-based initialization [24,17]; sparse sampling in large sparse problems [16]
An informed improvement phase does not close the gap	NSGAII-MIIP [17]
The cost of a sparse start: the large subsets are lost	Size sampling over a chosen range [30]; size-allocated and bounded starts [9,4]
Why the search stays near its start: unchanged children and repeats	Balanced mutation [21] and its unchanged expected size [17]; duplicate handling and caching [13,26,18]
Reporting recommendations	Evaluation budgets versus computing time in benchmarking [3]

at matched sizes, the dependence of reach on the number of features and the budget, the comparison with an informed improvement phase, and the counts of proposals and fits that explain the cost, across 46 datasets and up to three classifiers. The initialization, the mutation, and AUF are existing methods.

We test NSGA-II with the genetic operators of Li et al. [17], in our implementation, with and without their mutual-information improvement phase, so the results describe these configurations and the tested budgets and classifiers, not NSGA-II in general. Methods that change crossover or survival to keep small subsets [29,27,23,5,20] address the same problem by other means, and we do not compare against them. The returned front shows which subset sizes survive to the end of a run. It does not show which sizes were visited during the search, which would require a record of all evaluated subsets.

3. Problem Setting, Measures, and Analysis

3.1. Feature Selection with Two Objectives

A dataset has d features. A candidate subset is a binary vector $x \in \{0,1\}^d$, in which $x_j = 1$ means that feature j is selected, and its size $|x| = \sum_j x_j$ is the number of selected features. We train a classifier on the training part using only the selected features and measure its weighted F1 score on the validation part; we write this score as $s(x)$. The search minimizes $|x|$ and maximizes $s(x)$. A subset dominates another if it is no worse in both objectives and better in at least one. The empty subset cannot be fitted and receives $s = -1$, below every F1 score. Because no other subset is smaller, it is never dominated and can stay in the population, but it is not a usable answer: we define

the *returned front* as the non-empty subsets that no other member of the final population dominates. Each of them is fitted once more on the training part, with the same fixed random state and therefore the same model, and scored on validation and test. These final fits are not charged to the search budget.

3.2. Area Under the Front

We summarize a returned front with the area under the front (AUF) introduced with FASTENER [15]. For a limit of k features, let $q(k)$ be the test F1 of the subset that has the highest validation score among the subsets on the front with at most k features. If the front contains no subset with at most k features, $q(k) = 0$. AUF averages these values over the limits $1, \ldots, K$:

$$\mathrm{AUF}_K = \frac{1}{K} \sum_{k=1}^{K} q(k). \tag{1}$$

We use $K = 10$ as the primary limit and also report $K = 20$ and $K = d$, the AUF over the whole front. A subset with three features contributes to every limit $k \geq 3$, so a missing subset of exactly four features does not by itself give a zero at $k = 4$. Validation ties are broken deterministically: among subsets of the same size, the first one on the returned front is kept, and across sizes the smaller one. Such ties with different test scores occur at 2.0 to 4.8% of the reached limits in each group of runs. Because the subset is chosen on validation but scored on test, $q(k)$ need not increase with k; it decreases at some limit up to ten in 662 of the 1,544 main runs. AUF is therefore the mean of a validation-selected test curve, not the area under a monotone front. The value $q(k) = 0$ is assigned when the front holds no subset within the limit. A returned subset could also have a test F1 of zero, but no reached limit in our runs does, so every zero in our results marks an absent subset. The next subsection separates reach from quality.

3.3. Reach and Quality at Matched Sizes

We say that a run *reaches* the limit k if its returned front contains a subset with at most k features. For one configuration on one dataset, let $r(k)$ be the share of finished seeds whose run reaches k, and let $\bar{c}(k)$ be the mean of $q(k)$ over those seeds; $\bar{c}(k)$ is undefined when $r(k) = 0$. We report $R = r(K)$, the share of runs that reach ten features. Averaged over seeds, AUF is then $\overline{\mathrm{AUF}} = K^{-1} \sum_k r(k)\, \bar{c}(k)$: it combines how often a limit is reached with how well the returned subsets predict when it is.

To state how much of the difference $\Delta = \overline{\mathrm{AUF}}_a - \overline{\mathrm{AUF}}_b$ between configurations a and b on one dataset comes from reach, we use the symmetric split of a difference of products:

$$\Delta = \underbrace{\frac{1}{K} \sum_{k=1}^{K} \big(r_a(k) - r_b(k)\big) \frac{\tilde{c}_a(k) + \tilde{c}_b(k)}{2}}_{\text{reach part } \rho} + \underbrace{\frac{1}{K} \sum_{k=1}^{K} \frac{r_a(k) + r_b(k)}{2} \big(\tilde{c}_a(k) - \tilde{c}_b(k)\big)}_{\text{quality part } \kappa}, \tag{2}$$

where $\tilde{c}_a(k) = \bar{c}_a(k)$ if $r_a(k) > 0$, $\tilde{c}_a(k) = \bar{c}_b(k)$ if only b reaches k, and $\tilde{c}_a(k) = 0$ if neither does, and likewise for b. A limit that only one configuration reaches thus

contributes only to ρ, and a limit that neither reaches contributes nothing to either part. This is an accounting convention: where one configuration returns no subset, nothing is known about how well it would predict there, and the convention does not estimate it. The reach part includes limits that b reaches in fewer seeds than a, not only limits that it never reaches. Over the datasets i of a group, we report two summaries:

$$S = \frac{\sum_i \rho_i}{\sum_i \Delta_i}, \qquad A = \frac{\sum_i |\rho_i|}{\sum_i |\rho_i| + \sum_i |\kappa_i|}. \tag{3}$$

S is the share of the net AUF difference due to reach. It can exceed one when quality parts are negative. A is the share of the combined absolute size of both parts and lies between zero and one. Either summary is undefined when its denominator is zero, which does not occur in our groups. Both get 95% bootstrap intervals over datasets; resamples with a zero denominator are skipped.

To compare quality without the effect of reach, we compare a and b only at limits that both reach. Let M be the set of limits $k \le K$ with $r_a(k) \ge 0.6$ and $r_b(k) \ge 0.6$. Because r is a share of finished seeds, this means three of five seeds when all five finished, three of four, or two of three; every dataset with a matched limit in our results has five finished seeds for both starts. The *matched-size difference* on a dataset is $m = |M|^{-1} \sum_{k \in M} \big(\bar{c}_a(k) - \bar{c}_b(k)\big)$, undefined when M is empty. The two means in it may come from different seeds, so we also report a seed-paired version: for each seed s, it averages $q_{a,s}(k) - q_{b,s}(k)$ over the limits in M that both runs with that seed reach, and then averages over seeds. We repeat the comparison with subsets of exactly k features to check that the result does not depend on the at-most-k convention. These quantities are accounting steps for reporting results, not new measures.

A run's returned front is not the same as everything the run evaluated. A small subset can be evaluated, kept for some generations, and later replaced in the population. We therefore read the time at which a run first reaches the limit from its archive, the best subset of each size among all evaluated subsets, and we read reach at the end from the returned front. In all our runs, the archive and the returned front agree at the end of the run on whether the limit of ten features is reached.

3.4. Access to Small Subsets at Initialization

Whether the initial population already contains a small subset depends on how it is built. The opposition-based initialization used in NSGAII-MIIP [17] builds 50 pairs for a population of 100. The first member of a pair selects each feature independently with probability one half, and the second selects exactly the features the first does not. If the first member has m features, m follows a binomial distribution with d trials and probability one half, and its partner has $d - m$. A pair contains a non-empty subset with at most K features with probability

$$p_{\mathrm{pair}} = \Pr\left(1 \le m \le K \text{ or } 1 \le d - m \le K\right), \qquad m \sim \mathrm{Binomial}(d, \tfrac{1}{2}), \tag{4}$$

and the population contains one with probability $1 - (1 - p_{\mathrm{pair}})^{50}$. For $K = 10$, this probability is 1 for 16 and 24 features, 0.43 for 36, 0.11 for 40, and below 0.01 from 46 features on. Beyond a few dozen features, the opposition-based search must therefore produce every small subset through crossover, mutation, and selection.

The sparse initialization draws each subset size uniformly from 1 to $\min(30, d)$ and then selects that many features at random, the size sampling described by Miyandoab et al. [30] with a narrower range. For $d \geq 30$, each member has at most ten features with probability $1/3$, so a population of 100 lacks such a member with probability $(2/3)^{100} \approx 2.5 \times 10^{-18}$. On the two datasets with fewer features, PenDigits (16) and ItalyPowerDemand (24), the range is cut at d and the probability is $10/d$, larger than $1/3$. The sparse start therefore reaches the limit at the start, whatever the number of features.

The pilot of Section 4.4 adds two starts that cover all sizes from 1 to d. The *uniform start* draws the size uniformly from 1 to d, as in the initial population of Miyandoab et al. [30]; a member then has at most K features with probability K/d. The *log-uniform start* draws size k with probability proportional to $1/k$; a member has at most K features with probability H_K/H_d, where $H_n = \sum_{i=1}^{n} 1/i$. For $K = 10$ and a population of 100, the uniform start contains such a member with probability 0.997 for 176 features, 0.75 for 720, and 0.18 for 5,000, while for the log-uniform start the probability per member is still 0.32 at 5,000 features, so the population contains one with near certainty.

3.5. How Often Mutation Changes a Subset

The balanced mutation of NSGAII-MIIP [21,17] removes each selected feature with probability μ and adds each unselected feature with probability $\mu k/(d-k)$, where k is the current subset size. With the rate $\mu = 1/d$ used here and $0 < k < d$, the expected numbers of removed and added features are both k/d:

$$\mathrm{E}[\text{removed}] = k \cdot \frac{1}{d} = \frac{k}{d}, \qquad \mathrm{E}[\text{added}] = (d-k) \cdot \frac{k}{d(d-k)} = \frac{k}{d}. \tag{5}$$

The expected change in size is therefore zero, as intended, and the expected number of changed features is $2k/d$. For a small subset in a wide dataset, this is very small. Because each feature changes independently with a small probability, the number of changed features is close to a Poisson count with mean $2k/d$, and a child is left unchanged by mutation with probability close to $e^{-2k/d}$. With $d = 10{,}000$ and $k = 10$, the expected number of changes is 0.002, and a child is left unchanged with probability 0.998. For comparison, ordinary bit-flip mutation at the same rate changes one feature on average and changes the expected size by $1 - 2k/d$. At the boundaries, a full subset ($k = d$) can only lose features, each with probability $1/d$, and an empty one ($k = 0$) never changes. No repair step follows crossover or mutation, so a child can be empty; it is then scored as in Section 3.1. In the pilot, 0.03 to 7% of the proposed subsets per configuration were empty.

Crossover cannot compensate when the parents are similar. The single-point crossover used here exchanges the parents' tails after a position at which they differ, so the two children together keep the parents' total number of selected features, and two identical parents produce identical children. Once a sparse population contains many copies of a few small subsets, most children therefore repeat subsets that have already been evaluated. A repeated subset is taken from the cache and costs no model fit, so the run proposes many subsets for each new fit.

Repeats therefore have two sources: children that mutation leaves unchanged, and children that differ from their parents but equal a subset evaluated earlier. A mutation that always changes at least one feature removes the first source but not the second. Once the

Table 2. Datasets of the four experiments: ranges over the datasets of each. Rows are those of the training and test parts; the validation part has as many rows as the test part. Budget: model fits per run. EOData belongs to both the designed experiment and the benchmark, and the pilot's datasets come from the benchmark. The datasets are listed in Supplementary Table S7 and, for the confirmatory test, in Table 3

Experiment	Datasets	d	Training rows	Test rows	Classes	Budget
Designed	8	36–10,000	62–29,976	20–10,008	2–24	6,000–12,000
Benchmark	39	16–192,000	24–29,976	8–10,008	2–39	2,000–12,000
Pilot	4	176–720	275–591	93–196	2–37	1, 3, 10×[a]
Confirmatory	7	128–1,024	240–30,002	80–9,999	2–40	until reach[b]

[a] Multiples of the benchmark budget. [b] Each run stops once it reaches ten features (Section 4.5).

population has converged on a few small subsets, a single change to one of them often gives a neighbor that has already been evaluated. The pilot tests both parts with such a mutation (Section 4.4). This account follows from the operators; it describes a mechanism that the results in Section 5 are consistent with, not a proof that it is the only cause.

4. Experimental Design

4.1. Datasets

We use 46 classification datasets with 16 to 192,000 features and 2 to 39 classes (Table 2; each is listed with its domain, rows, classes, and budget in Supplementary Table S7). Eight of them come from an experiment designed for this comparison and were chosen to cover feature counts from 36 to 10,000. They are EOData, a land-cover dataset built from Sentinel-2 satellite observations [22] for which FASTENER was developed [15]; satimage and har from OpenML [25]; and USPS, COIL20, gisette, Prostate_GE, and arcene from the scikit-feature repository [19]. The other 38 datasets come from the UCR and UEA time series archives [7,2] and were run, together with EOData, in a broader benchmark of feature-selection methods. They were chosen to cover the range of dataset shapes in the archives: the archive datasets were grouped by the logarithms of their numbers of examples and features with k-means ($k = 7$), and three to seven datasets were taken from each group. Seven further datasets are used only in the confirmatory test of Section 4.5.

In the time series datasets, one row is one series, and its time points are the features. For a series with several channels, each pair of a channel and a time point is a feature. Neighboring time points are strongly correlated, so these datasets contain many redundant features, which is the situation feature selection is meant to address.

Every dataset is split once into stratified training, validation, and test parts in the proportions 60/20/20. The split is the same for all seeds and configurations. For the time series, we merge the training and test parts of the archive before splitting, so published accuracies on these datasets are not comparable with ours. EOData has 480,000 rows, of which we use a fixed stratified sample of 50,000. The search and the choice of the best subset at each size use only the validation part; the test part is used once, for the reported

scores. The split of EOData is random rather than spatial, because the published data contain no parcel identifiers. Neighboring pixels of one field can therefore fall into different parts, and its test scores are likely optimistic.

Each run has a fixed budget of model fits. A subset that was evaluated before is taken from a cache of stored results and does not count against the budget. In the designed experiment, the budget is 6,000 fits, and 12,000 on arcene. In the benchmark, the budget was set for each dataset before the runs, from one calibration run of the FASTENER method with a separate seed, read on the validation part only. Of the values 1,000, 2,000, 4,000, and 6,000, we took the smallest at which the calibration run's validation AUF was within 0.005 of its value at the end of the run (12,000 fits) and no more than 0.005 below that of a Boruta reference, and then the next larger value of 2,000, 4,000, 6,000, or 12,000. If no value met both conditions, the budget is 12,000. This gives 2,000 to 12,000 fits. The budgets therefore follow the convergence of a different search method, not of NSGA-II.

4.2. Search Configurations

We compare three configurations of NSGA-II [8]. All use the operators of NSGAII-MIIP [17] in our own Python reimplementation[1] of the authors' published MATLAB code. They differ only in the initial population and in whether the improvement phase runs:

- **Opposition-based (OBL) start**: the opposition-based initialization of NSGAII-MIIP (Section 3.4), without the improvement phase.
- **Sparse start**: subset sizes drawn uniformly from 1 to $\min(30, d)$, then that many features drawn at random, without the improvement phase.
- **NSGAII-MIIP**: our implementation of the complete method, the opposition-based start with the mutual-information improvement phase. It was run only on the 39 benchmark datasets.

Algorithm 1 gives one run. Crossover keeps the pair's total number of selected features; parents that differ in at most one position are copied. In NSGAII-MIIP, the improvement phase runs in every generation. It discretizes each feature of the training part into ten equal-width bins, estimates mutual information between features and with the class, groups the features into clusters, and adds, removes, or exchanges features according to their relevance to the class and their redundancy with the selected features [17]. The mutual information and clusters are computed once per dataset from the training part; their computing time is part of the elapsed time but costs no fits. Our implementation departs from the published method in four documented ways: the objective is the validation weighted F1 of the run's classifier rather than the geometric mean over internal cross-validation folds, the fit budget is ours, negative distances in the clustering are clipped at zero, and the empty subset is handled as above. We did not compare its outputs with those of the authors' code, so our results describe this implementation.

Every new subset costs one model fit, including those of the improvement phase, and a cached subset costs nothing, so a run can propose far more subsets than it fits. The budget is applied subset by subset in every scoring step, the improvement phase included: once it is spent, cached subsets are still scored, the others are dropped, and those scored earlier in the same batch are kept, so no run exceeds its budget. A run ends when its budget

[1] `https://github.com/klemenkenda/nsga2-fs`

Algorithm 1 One run of the tested NSGA-II configurations; B is the fit budget and $\mathcal{C}$ the cache

$P \leftarrow$ 100 subsets from the start (OBL, sparse, or, in the pilot, uniform or log-uniform)
score P ▷ a subset not in $\mathcal{C}$ costs one fit; empty subsets get $s = -1$
while fewer than B fits are spent **do**
    rank P by nondominated sorting and crowding distance
    choose 100 parents by binary tournament on (rank, crowding)
    with probability 0.9, cross each pair at a random differing position other than the first
    apply balanced mutation at rate $1/d$ to every child (forced flip in the pilot arms)
    score the children; once B fits are spent, drop children that would need a new fit
    if NSGAII-MIIP **then** improve the first front of $P \cup$ children with MIIP and score the results under the same cap
    end if
    $P \leftarrow$ best 100 of the union by rank, the last front cut by crowding distance
    if stopping rule on and 50 generations in a row added no fit **then stop**
    end if
end while
return the non-empty members of the first front of P, refitted and scored on validation and test

is spent. In the benchmark runs with the random forest and LightGBM, a run also ends after 50 consecutive generations without a new subset; without this rule, a run that only repeats subsets could continue for hundreds of thousands of generations. The runs with the decision tree have no such rule. Runs stopped by the rule spend less than their nominal budget (Section 5.6). Every benchmark run was limited to six hours, and runs that reached the limit are counted as missing. Twelve combinations of dataset and configuration were not run, because their estimated memory or time exceeded the available resources.

Each configuration is run with five seeds, 2020 to 2024, on every dataset. The seed changes the search, not the data split. In the designed experiment, the classifier is a decision tree. In the benchmark, every dataset and configuration is run three times, each time with a different classifier: a decision tree, a random forest with 100 trees, and LightGBM. We treat the three as separate groups of runs. The tree and the forest use the scikit-learn defaults. LightGBM uses its defaults except for the smallest number of rows in a leaf, which is a twentieth of the training rows, between 1 and 20, so that the small datasets can still be split. All three use a fixed random state, so the same subset always receives the same score. The classifier both guides the search and produces the reported scores.

4.3. Statistical Analysis

Datasets are the units of analysis. Seeds of the same dataset share its data split and are not independent samples of the population of datasets, so for each dataset and configuration we first average over seeds; reach becomes the share of seeds that reach the limit. We analyze each group of runs (the designed experiment and the three benchmark groups) separately.

To compare two configurations, we take their paired differences over the datasets on which both have finished runs. We report the numbers of wins, ties, and losses, the mean difference with a 95% percentile bootstrap interval over datasets (10,000 resamples), a two-sided Wilcoxon signed-rank test, and the matched-pairs rank-biserial correlation as

the effect size. Because we run several related tests, we correct their p-values with the Holm method [11], which controls the probability of at least one false finding within a family of tests and is less conservative than the Bonferroni correction. A result counts as significant if its corrected value is below 0.05. A family is a set of tests that answer one question: the four groups of runs for the main comparison of the two starts (Table 4), and the three pairs of configurations within a group for the comparisons among the three configurations. The comparison of the two starts belongs to both families, so its corrected p-values in Table 4 and in the supplementary pairwise table differ slightly; neither changes a decision. The supplementary material lists the intervals and effect sizes of the principal comparisons and the per-dataset results.

The matched-size difference (Section 3.3) is analyzed over every dataset on which it is defined. We call the mean difference of a group small if its 95% interval lies within ± 0.02 of zero, and inconclusive if the interval includes zero but extends beyond the margin. This statement concerns the group mean over these datasets, not each dataset, so we also report the individual datasets and the seed-paired version. A group with a single such dataset has no interval, and we draw no conclusion about it. After reading this analysis, we added an exploratory stability check without the datasets with at most 50 test rows: BeetleFly (8 test rows), Coffee (11), OliveOil (12), BasicMotions (16), Prostate_GE (20), arcene (40), and Ham (43). Very small test sets motivated this check, but the cutoff of 50 rows is not a validated precision threshold, and we did not estimate the uncertainty of F1 differences on individual test sets. Because the rule was chosen after the result was known, the analysis without it is primary.

For the time to reach, we use the share of the fit budget spent when a run's archive first holds a subset with at most ten features. A run that never reaches the limit is censored at its last fit count. In all completed runs this is the full budget, because every run stopped early by the stagnation rule had already reached the limit; the measure is thus a budget-normalized description of the completed runs. We show Kaplan–Meier curves of the share of runs that have reached the limit and compare configurations by the restricted mean time to reach, up to the full budget, per dataset. To relate reach to the number of features, we fit a logistic regression of reach on $\log_2 d$ and $\log_2$ of the budget over runs, and report the number of features at which half of the runs reach the limit at the median budget, with a 95% bootstrap interval over datasets. This is a descriptive location for these datasets and budgets, not a general threshold.

Runs that did not finish are left out of the main analysis. Missing runs are concentrated in the OBL start and NSGAII-MIIP on wide datasets (Supplementary Section S3), so they may be informative. We therefore repeat every comparison in three ways: with only the datasets on which all five seeds finished, with every missing run counted as reaching nothing and scoring zero, and with only the datasets on which every configuration has runs. These are three scenarios, not bounds on what the missing runs would have shown. To compare the classifiers, we use the 26 datasets on which both starts have runs under all three, with a Friedman test and Holm-corrected pairwise Wilcoxon tests. The five seeds per dataset share one split, so they measure the variability of the search, not of the split. They enter only through these averages, and most datasets come from one archive, so the tests describe these datasets and should not be read as estimates for feature selection in general.

4.4. Pilot Experiment

After the main analysis, we ran a pilot to examine three questions it raised: whether a start that covers all subset sizes keeps the reach of the sparse start without losing the large subsets, whether the opposition-based start's lack of reach is a matter of budget, and whether a mutation that always changes a subset removes the repeats. We registered its criteria before the runs. The pilot uses four datasets from the range of feature counts in which reach changes (Section 5.2): Adiac (176 features), Strawberry (235), Earthquakes (512), and Computers (720), each with at least 90 test rows. Their earlier results had been seen, so the pilot guides the interpretation and does not confirm it. It uses the decision tree, five new seeds (2025 to 2029), the benchmark budgets (6,000 fits on Strawberry, 12,000 on the others), and the stopping rule of 50 generations without a new subset in every run.

The pilot compares ten configurations of NSGA-II without the improvement phase. Four starts are combined with two mutations: the opposition-based, sparse, uniform, and log-uniform starts of Section 3.4, each with the balanced mutation and with a *forced flip*. The forced flip applies the balanced mutation and then changes a child that it left unchanged by one feature: for $1 < k < d$ it adds a random unselected feature or removes a random selected one with equal probability, so the expected change in size of these forced moves is zero; for $k \leq 1$ it can only add, and for $k = d$ only remove. An unchanged empty child therefore gains one feature, while a child that the balanced mutation itself emptied stays empty and is scored as in Section 3.1. The two other configurations give the opposition-based start three and ten times the budget. The $1\times$ and $3\times$ runs share their seeds and random draws with the $10\times$ runs, so the three budgets are points on the same search trajectories, not independent runs. Configurations that differ only in mutation start from the same initial population, which we checked for every pair. Every run records each evaluated subset and, per generation, the numbers of proposals, cache hits, and children left unchanged by mutation.

A comparison holds on a dataset if it holds in at least four of five seeds, paired by seed, and a part of a criterion holds if it holds on at least three of the four datasets. Four datasets are too few for a test over datasets.

4.5. Confirmatory Test of the Budget

The pilot showed that the opposition-based start reaches ten features if it is given enough fits (Section 5.7). To find out how many it needs, we measured the fits until its archive first held a subset with at most ten features in the 20 pilot runs with ten times the budget. A least-squares line through the logarithms of these fits and of d gives the prediction

$$F(d) = e^{-0.961}\, d^{\,1.877} \approx 0.38\, d^{\,1.88}. \tag{6}$$

For example, $F(500) \approx 44{,}000$ fits. The pilot's four datasets cannot fix the exponent well, so we registered this prediction and its criteria before testing it on seven datasets on which no NSGA-II configuration had been run: gas_drift, volkert, and gina from OpenML [25], crop_mapping and madelon from the UCI repository [14], and Isolet and ORL from scikit-feature [19] (Table 3). None of them is a time series. They come from chemical sensing, crop mapping from satellite data, speech, handwritten digits, face images, an anonymized benchmark, and synthetic data, and they have 128 to 1,024 features, 2 to 40 classes, and

Table 3. Datasets of the confirmatory test and the fits the opposition-based start needed before its archive first held a subset with at most ten features. Predicted: the line fitted on the pilot, registered before the runs; observed: median and range over five seeds; ratio: median over prediction

Dataset	d	Training rows	Classes	Fits to reach ten features			Ratio
				Predicted	Observed	Range	
gas_drift	128	8,346	6	3,444	5,276	3,251–6,109	1.53
crop_mapping	174	30,002	7	6,127	9,758	8,389–11,975	1.59
volkert	180	30,002	10	6,530	7,986	7,602–12,213	1.22
madelon	500	1,560	2	44,412	54,892	33,916–75,882	1.24
Isolet	617	936	26	65,896	70,482	63,906–132,956	1.07
gina	970	2,080	2	154,019	183,226	153,828–202,233	1.19
ORL	1,024	240	40	170,501	162,597	153,897–214,244	0.95

more than 50 test rows each. They are split as in Section 4.1, with crop_mapping and volkert sampled to 50,000 rows.

The test uses the opposition-based start of the pilot with the decision tree, five new seeds (2030 to 2034), and the stopping rule. A run ends after the generation in which it first scores a subset with at most ten features, or at a cap of $4F(d)$ fits. Stopping early does not change the measurement: in the pilot, the $1\times$ and $3\times$ runs were exact beginnings of the $10\times$ runs, so the fits to reach the limit do not depend on the budget. The fits are counted at the end of the generation, so they can be up to 100 too high. A run that does not reach the limit before the cap, the stopping rule, a 24-hour limit, or a failure is censored. The test has three parts:

1. on every dataset, at least four of five runs reach the limit before the cap;
2. the median over the five runs lies within a factor of two of $F(d)$ on at least five of the seven datasets;
3. the least-squares slope of the logarithm of the median on $\log d$ has a 95% bootstrap interval over datasets (10,000 resamples) whose lower end is above 1, so that the fits grow faster than in proportion to d.

The prediction holds if all three parts hold. The first attempt was stopped by the check against uncommitted code before any run started and was repeated unchanged.

4.6. Implementation and Reproducibility

The code, run logs, derived tables, and a reproduction guide are publicly available; Supplementary Section S2 gives the repositories, the software environment, and the recorded code commits.

5. Results

5.1. The AUF Difference Is a Difference in Reach

With a limit of ten features, the sparse start has a much higher AUF than the opposition-based (OBL) start: in the three benchmark groups, the mean difference per dataset is 0.42

Table 4. Sparse start against the opposition-based (OBL) start, by group of runs, with datasets as units and 95% bootstrap intervals over datasets. (a) Reach: share of seeds whose front holds a subset with at most ten features; wins/ties/losses, mean difference, and Wilcoxon p, Holm-corrected over the four groups. S and A: share of the AUF difference due to reach, signed and absolute (Eq. 3), in percent. (b) Matched-size difference m in test F1 (Section 3.3): number of datasets with a matched limit, mean, and datasets with $|m| > 0.02$; the same without datasets with at most 50 test rows (exploratory); and the mean of the seed-paired version over all datasets

(a) Reach and its share of the AUF difference

Group	Data	W/T/L	Mean [95% CI]	p	S [95% CI]	A [95% CI]
Decision tree (designed)	8	7/1/0	0.85 [0.60, 1.00]	0.016	100.1 [99.9, 100.5]	99.8 [99.4, 100.0]
Decision tree	30	22/8/0	0.68 [0.51, 0.84]	1.8×10^{-5}	100.0 [99.8, 100.2]	99.7 [99.4, 99.9]
Random forest	34	24/10/0	0.64 [0.47, 0.78]	1.2×10^{-5}	101.2 [99.5, 104.3]	98.1 [95.2, 99.7]
LightGBM	28	17/11/0	0.54 [0.36, 0.72]	2.1×10^{-4}	100.0 [99.3, 100.8]	99.2 [98.3, 99.8]

(b) Test F1 at matched sizes, sparse minus OBL start

	All datasets (primary)			>50 test rows		Paired
Group	Data	Mean [95% CI]	$\|m\|>0.02$	Data	Mean [95% CI]	Mean
Decision tree (designed)	1	−0.007	0	1	−0.007	−0.007
Decision tree	10	−0.001 [−0.004, +0.002]	0	10	−0.001 [−0.004, +0.002]	−0.002
Random forest	13	−0.014 [−0.044, +0.008]	4	11	+0.006 [+0.000, +0.013]	−0.015
LightGBM	13	+0.001 [−0.006, +0.008]	2	11	+0.003 [−0.002, +0.011]	−0.000

to 0.49. Table 4 shows where this difference comes from. On no dataset in any group does the OBL start reach the limit in more seeds than the sparse start. The ties are datasets with few features, on which both starts always reach it. The mean difference in reach is 0.54 to 0.85, with Holm-corrected $p \leq 0.016$ and rank-biserial correlation 1 in every group.

Splitting each dataset's AUF difference into a reach part and a quality part (Eq. 2; Fig. 1) shows that the split is dominated by reach. The two summaries measure different things. The reach part is 100.0 to 101.2% of the net AUF difference (S in Table 4), and 98.1 to 99.8% of the combined absolute size of the reach and quality parts (A); the lowest lower bound of the intervals is 95.2%, for A under the random forest. Most of the reach part comes from limits that the OBL start never reaches on a dataset: they account for 88 to 99% of the net difference, and the rest comes from limits that it reaches in fewer seeds than the sparse start. The quality parts lie close to zero on almost every dataset. The two clear exceptions, below zero under the random forest, are BeetleFly and Coffee, the datasets with 8 and 11 test rows, which is why S exceeds 100% there.

Sizes that both starts reach occur on only 1 to 13 datasets per group, with at most 512 features (Table 4b). Under the decision tree and LightGBM, the mean difference in test F1 at matched sizes is -0.001 and $+0.001$, with 95% intervals within ± 0.02. Under the random forest, it is -0.014 with the interval $[-0.044, +0.008]$, which exceeds the margin, so the primary analysis is inconclusive for this classifier. BeetleFly (-0.106) and Coffee (-0.138), both in favor of the OBL start, cause this. Without the datasets with at most 50 test rows, which leaves 11 datasets with at most 235 features, the mean is $+0.006$

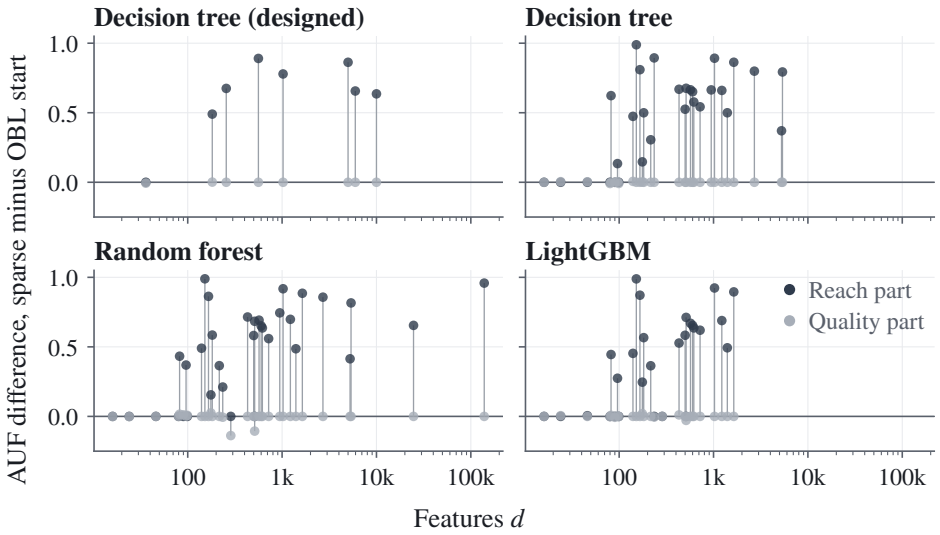


Fig. 1. The split of the AUF difference between the two starts is dominated by reach. Each pair of points joined by a line is one dataset: the part of the sparse start's AUF lead due to reaching more limits (reach part) and the part due to better scores at the limits both reach (quality part). Test split, seed means, limit of ten features

$[+0.000, +0.013]$. This rule was chosen with the primary result in view, so the result is an exploratory stability check, not a confirmation. Besides BeetleFly and Coffee, individual datasets differ more than the group means (Supplementary Table S3): Adiac ($+0.028$) and TwoLeadECG ($+0.022$) under the random forest and Adiac ($+0.036$) under LightGBM exceed 0.02 in favor of the sparse start. The seed-paired means differ from the conditional means by at most 0.0011 in the primary analysis and 0.0013 in the exploratory one, so they do not change the interpretation; subsets of exactly k features give the same picture. The designed experiment has a single matched dataset, satimage (-0.007), from which no general statement follows. Nothing is known about quality at matched sizes on wider datasets, because there the OBL start returns no subset to compare.

5.2. Reach Depends on the Number of Features

Figure 2 relates reach to the number of features. The initial opposition-based population holds a subset with at most ten features only on PenDigits (16 features), ItalyPowerDemand (24), and in one seed of satimage (36), as the analysis of Section 3.4 predicts. Selection then brings the front to the limit within the budget on every dataset with at most 99 features. Between 152 and 512 features, some datasets reach the limit and others do not, and beyond 512 features none does. A logistic model puts the number of features at which half of the OBL runs reach the limit at 163 $[122, 202]$, 212 $[145, 277]$, and 226 $[148, 300]$ under the three classifiers of the benchmark and at 172 $[92, 174]$ in the designed experiment. For NSGAII-MIIP, the same point lies at 208 $[159, 253]$, 268 $[224, 343]$, and 266 $[210, 302]$ features. These are fitted locations for these datasets and budgets, not thresh-

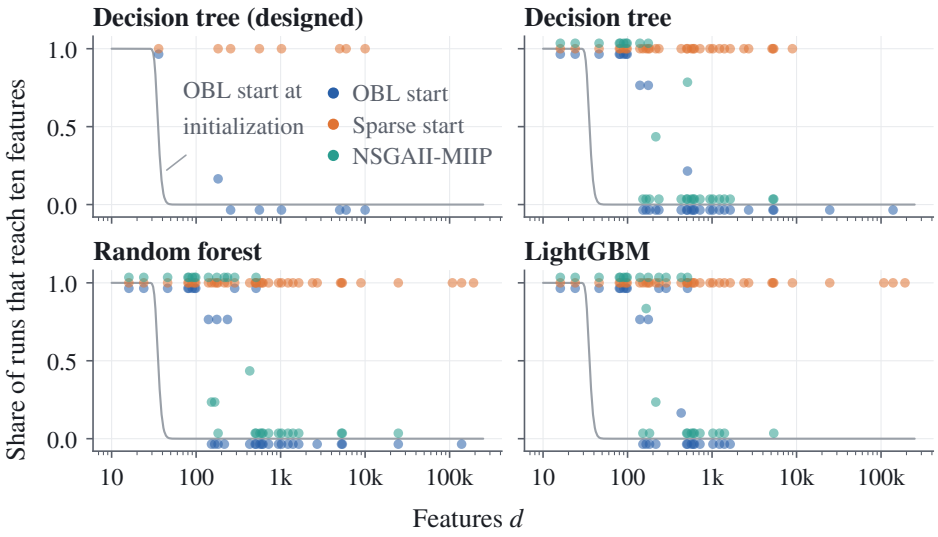


Fig. 2. The opposition-based search reaches ten features on datasets with up to about 100 features and almost never beyond 500, while the sparse start always does. Each point is one dataset: the share of its five runs whose returned front holds a subset with at most ten features; points are offset slightly by configuration. The grey curve is the probability that an opposition-based initial population already holds such a subset (Eq. 4)

olds that carry over to other benchmarks. The budgets were set per dataset and grow with the number of features, so the model cannot separate the two, and the intervals for the budget coefficient are wide. We therefore do not claim a separate effect of the budget from these runs; the pilot and the confirmatory test vary the budget directly (Sections 5.7 and 5.8).

The sparse start reaches the limit in all 594 of its runs. Every one of its returned fronts holds a subset with a single feature.

5.3. When the Limit Is Reached

Figure 3 shows when runs first reach the limit. All runs of the sparse start reach it with the initial population. Of the opposition-based runs, 12.5%, 6.7%, 6.5%, and 9.0% have reached it after 10% of the budget, in the designed experiment and under the decision tree, random forest, and LightGBM, and 15%, 30%, 36%, and 48% by the end. NSGAII-MIIP reaches it in 37%, 46%, and 56% of its runs by the end. The restricted mean time to reach is shorter for the sparse start than for the OBL start on every dataset (rank-biserial correlation -1), by 0.72 to 0.86 of the budget on average, and shorter than for NSGAII-MIIP by 0.66 to 0.72. No run lost a small subset once its archive held one: in all 1,544 runs, the returned front reaches the limit exactly when the archive does.

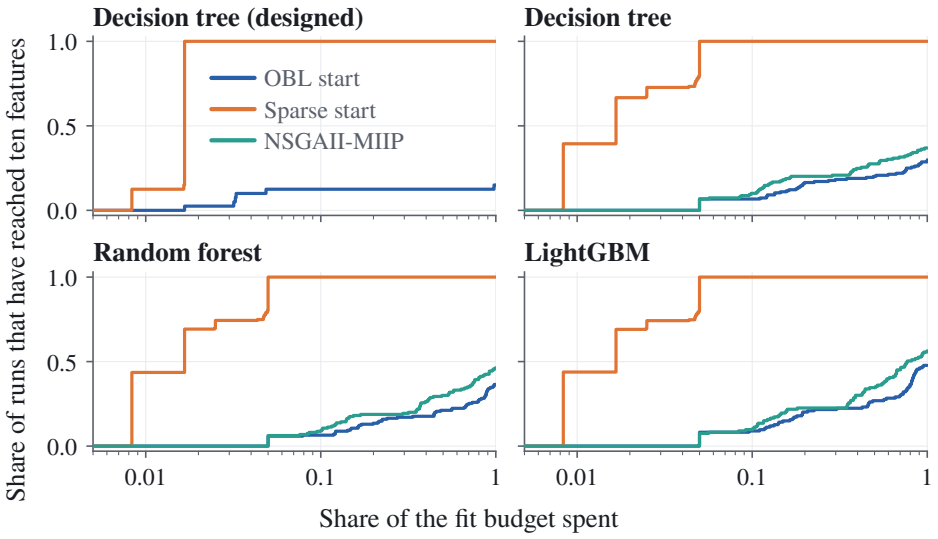


Fig. 3. The sparse start holds a subset with at most ten features from its first generation, while most opposition-based runs never reach one. Kaplan–Meier curves of the share of runs whose archive holds such a subset, against the share of the fit budget spent (logarithmic). Runs within a group are pooled for display; the tests in the text are per dataset

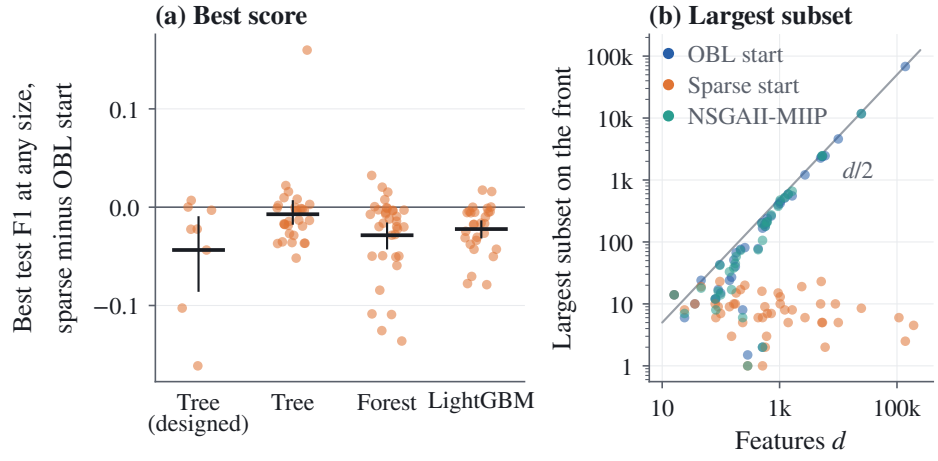


Fig. 4. The sparse start gives up the large subsets. (a) Per dataset, the best test F1 at any size of the sparse start minus that of the opposition-based start, by group (tree: decision tree; forest: random forest); bars are means, vertical lines their 95% bootstrap intervals. (b) The largest subset on the returned front, median over seeds and groups, with $d/2$ for reference. Test split

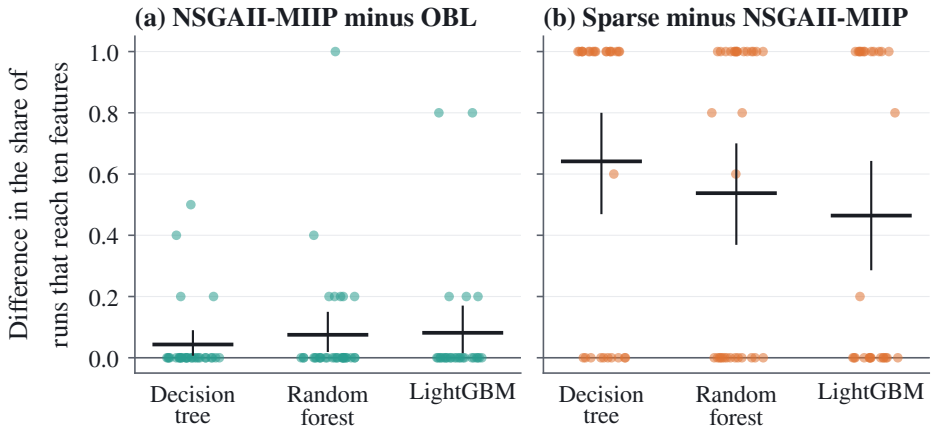


Fig. 5. The improvement phase of our NSGAII-MIIP implementation adds little reach. Per dataset, the difference in the share of runs that reach ten features: (a) NSGAII-MIIP minus the opposition-based start, (b) the sparse start minus NSGAII-MIIP; bars are means, vertical lines their 95% bootstrap intervals

5.4. The Cost of a Sparse Start

The sparse start does not search better in general; it searches elsewhere. Without a size limit, the best subset on its front, chosen on validation, has a lower test F1 than the best subset of the opposition-based start on most datasets (Fig. 4a). The mean difference is -0.007 under the decision tree (7 wins, 23 losses; Holm $p = 0.013$), -0.029 under the random forest (5 wins, 29 losses; $p < 0.001$), and -0.022 under LightGBM (3 wins, 1 tie, 24 losses; $p < 0.001$). Against NSGAII-MIIP the difference is -0.012 to -0.021. In the designed experiment, the sparse start has the lower best score on five of the eight datasets.

The reason is visible in the sizes (Fig. 4b). The fronts of the sparse start end at no more than 36 features on any dataset, while those of the opposition-based start and of NSGAII-MIIP reach close to half of all features on the wide datasets. The median difference in the largest subset is 97, 69, and 28 features under the three classifiers. The sparse start still leads at larger limits: its AUF is higher by 0.40 to 0.48 with a limit of 20 features, and by 0.08 to 0.14 when AUF is taken over the whole front ($K = d$). The pilot shows that a start covering all sizes does not remove this cost (Section 5.7).

5.5. An Informed Improvement Phase Does Not Close the Gap

Our implementation of NSGAII-MIIP starts from the same OBL population and adds the mutual-information improvement phase. Compared with the opposition-based start alone, its share of runs that reach the limit is higher by 0.043, 0.075, and 0.082 under the three classifiers (Holm $p = 0.125, 0.016, 0.062$), and its AUF by 0.058, 0.057, and 0.083 ($p = 0.005, 0.031, 0.029$). The sparse start is ahead of it by 0.64, 0.54, and 0.46 in reach and by 0.44, 0.39, and 0.33 in AUF (Fig. 5). In these runs, the improvement phase

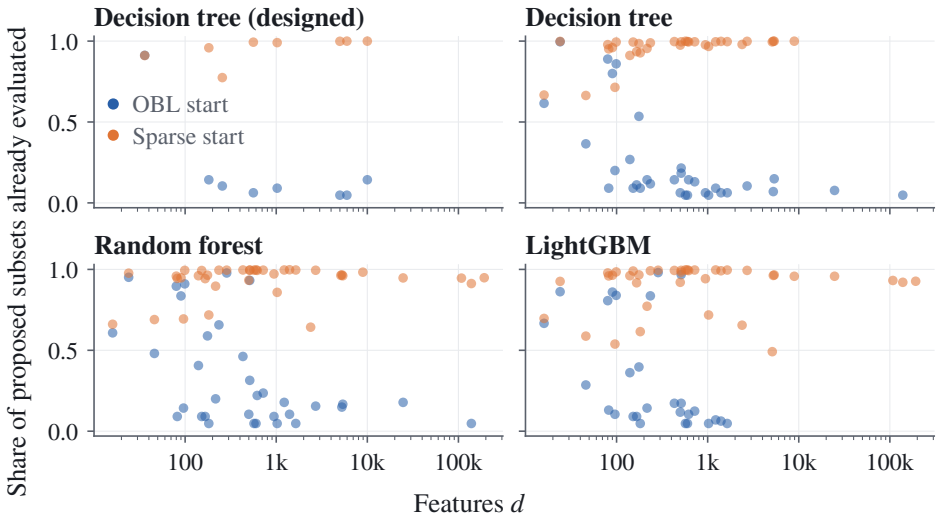


Fig. 6. Almost all subsets proposed in sparse runs had been evaluated before. Per dataset, the median over seeds of the share of proposed subsets that were taken from the cache, against the number of features. NSGAII-MIIP is not shown, because its improvement phase evaluates subsets outside the generation count

therefore helps slightly, while the start matters far more for whether the front reaches small sizes.

5.6. Repeated Subsets

The sparse start spends its proposals differently (Fig. 6). Over all runs of a group, the median share of proposed subsets that had already been evaluated is 0.96 to 0.99 for the sparse start and 0.10 to 0.18 for the OBL start, depending on the group. This agrees with the analysis of Section 3.5: in a population of small subsets, mutation rarely changes a child, and crossover between similar parents rarely produces a new subset. The pilot measures the first part directly (Section 5.7). Under the random forest and LightGBM, the stopping rule of 50 generations without a new subset ended 72 of 195 and 63 of 194 sparse runs, at a median of 8.8% and 7.9% of the budget, and 10 of 170 and 9 of 134 opposition-based runs, at 75% and 59%. Every stopped run had already reached the limit. The runs with the decision tree have no such rule, but they also differ in classifier, hardware, and code version, so the comparison between groups does not isolate the effect of the rule.

5.7. Pilot: Budget, Starts over All Sizes, and a Forced Flip

The pilot (Section 4.4) did not support any of its three registered claims, but each result sharpens how the main results should be read.

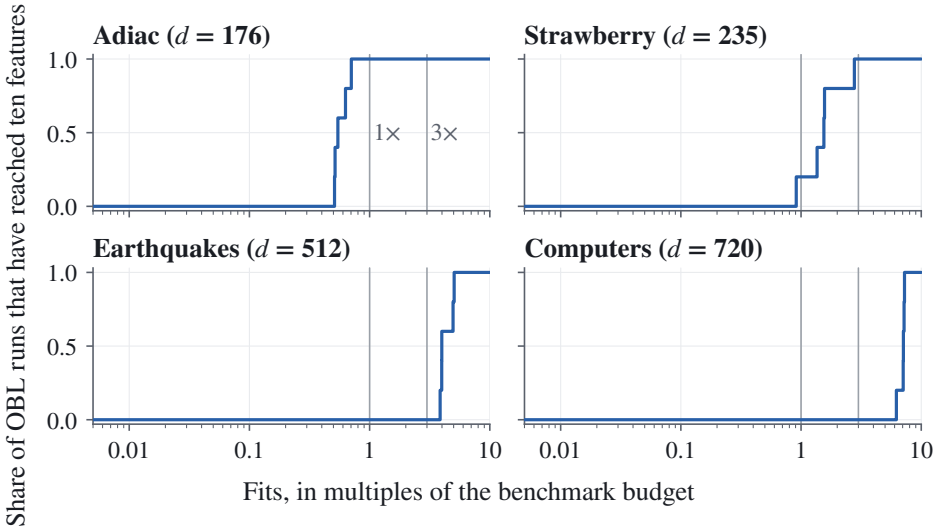


Fig. 7. With enough fits, the opposition-based start reaches ten features on every pilot dataset; the fits it needs grow with the number of features. Kaplan–Meier curves of the share of the five 10× runs whose archive holds a subset with at most ten features, against fits in multiples of the benchmark budget. The 1× and 3× runs are the same trajectories cut short. Validation split

The opposition-based start reaches the limit with more budget. The share of opposition-based runs whose front reaches ten features at 1×, 3×, and 10× the budget is 1.0, 1.0, and 1.0 on Adiac, 0.2, 1.0, and 1.0 on Strawberry, and 0.0, 0.0, and 1.0 on both Earthquakes and Computers (Fig. 7). The registered expectation that reach would stay incomplete at 10× on the two widest datasets therefore fails. The archive first holds a subset with at most ten features after a median of 6,534, 9,292, 47,830, and 85,239 fits, that is, 0.54, 1.55, 3.99, and 7.10 times the benchmark budget, or 37, 40, 93, and 118 fits per feature. The effort grows with the number of features on these four datasets, and so do the fits per feature, but four datasets that also differ in other respects cannot show how it scales; the confirmatory test does (Section 5.8). Once it gets there, the OBL start scores similarly to the sparse start: at 10× its AUF is 0.009 higher than that of the sparse start at 1× (higher in 12 of 20 seeds), at a median run time of 1,480 to 4,691 seconds against 216 to 907. On these four datasets, the boundary of Section 5.2 is thus one of budget: at a fixed budget, AUF with a small limit largely measures how far the start is from the small sizes.

Starts over all sizes keep reach but not the large subsets. Both starts that cover sizes 1 to d reach ten features in every run. Their best test F1 at any size is at least that of the sparse start in only 7 and 8 of 20 seeds (mean differences -0.014 and -0.005), and their AUF differs from it by 0.000 and -0.002 on average (Fig. 8). The reason is in the fronts (Table 5). Whatever the start, the population ends on a few small subsets, and the large subsets that a wide start contains at the beginning do not survive. With ten times the budget, the OBL start also ends on small subsets once it has reached them. The loss at the

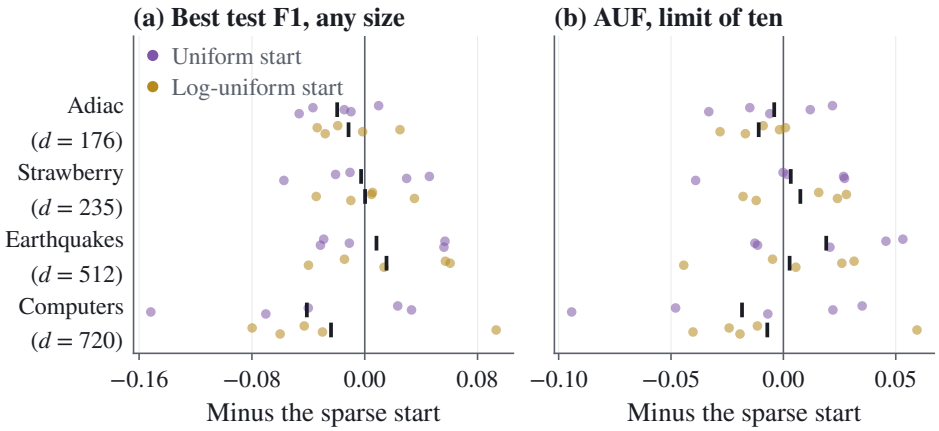


Fig. 8. Starts that cover all sizes score similarly to the sparse start, not better. Per dataset and seed, the uniform and log-uniform starts minus the sparse start with the same seed: (a) best test F1 at any size, (b) AUF with a limit of ten features; bars are means. Test split

Table 5. Largest subset on the returned front in the pilot, median over five seeds. Whatever the start, the search ends on a few small subsets once it has reached them

Start	Adiac ($d = 176$)	Strawberry ($d = 235$)	Earthquakes ($d = 512$)	Computers ($d = 720$)
Sparse start	10	6	7	6
Uniform start	11	8	13	8
Log-uniform start	9	6	8	5
OBL start	26	34	165	269
OBL start, 10× budget	18	6	13	20

large-subset end (Section 5.4) therefore arises during the search under the tested variation and survival rules, not from the start. We did not vary selection or crossover, so the pilot does not show which step removes the large subsets.

Repeats follow the $2k/d$ account, and a forced flip lowers their cost but not the quality. The share of children that balanced mutation leaves unchanged lies close to $e^{-2k/d}$ (Supplementary Fig. S3): 0.94 to 0.99 per dataset for the three starts with small subsets and 0.48 to 0.85 for the opposition-based start. The forced flip lowers the share of repeated proposals in every seed, from a median of 0.94 to 0.73 for the sparse start and from 0.94 to 0.70 for the log-uniform start, but never by half, and it changes AUF little (-0.003 and 0.000 on average; Supplementary Fig. S4). With the forced flip, a child rarely equals a member of the population (5% of proposals, against 45% with balanced mutation for the sparse start), yet 78% of all proposals had already been evaluated: the second source of repeats of Section 3.5. What the forced flip changes is the cost. Pooled over its runs, the sparse start makes 4.7 proposals per new fit instead of 176, and its median run time on the four datasets falls from 290, 216, 847, and 907 seconds to 106, 28, 52, and 54. For the

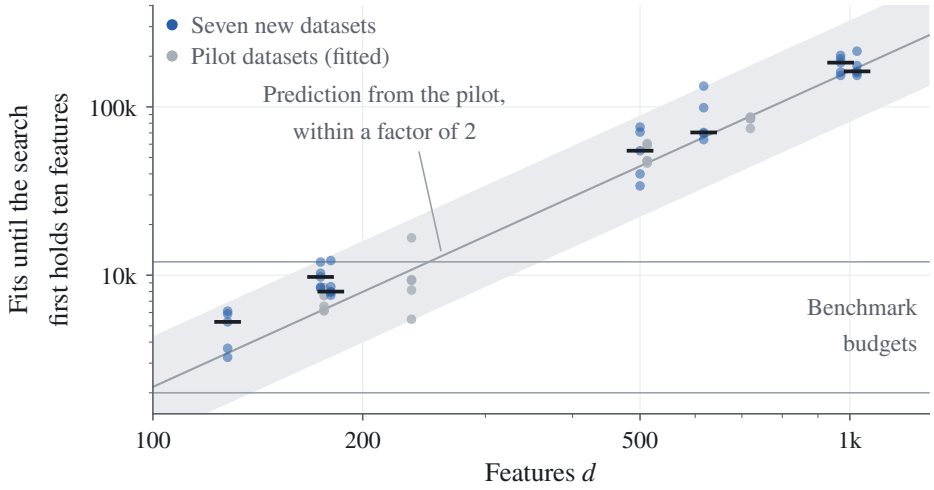


Fig. 9. On seven new datasets, the fits the opposition-based start needs to reach ten features grow faster than the number of features, as predicted from the pilot. Fits until the archive first holds a subset with at most ten features, one point per run; bars are medians over five seeds. The line is the registered prediction (Eq. 6), shaded within a factor of two; the horizontal lines mark the smallest and largest benchmark budgets. Validation split

opposition-based start, whose population is still wide, the forced flip removes almost all repeats and raises AUF by 0.016.

5.8. Confirmatory Test: The Fits Needed to Reach Ten Features

The prediction of Section 4.5 holds on all three registered parts (Table 3, Fig. 9). All 35 runs reached ten features before the cap, none after more than half of it. The medians lie between 0.95 and 1.59 times the prediction, so all seven datasets are within the factor of two; of the single runs, only one, on Isolet, falls just outside it (2.02 times). The slope over the seven datasets is 1.70 $[1.61, 1.82]$, well above 1, so the fits grow faster than in proportion to d. The interval does not include the predicted exponent of 1.88, which was estimated on four datasets. The prediction was too low on six of the seven datasets, by a factor of 1.24 on geometric average, most on the datasets with the fewest features (1.53 to 1.59 at 128 to 174 features) and least on those with the most (0.95 to 1.19 at 970 to 1,024). Over the eleven datasets of the pilot and the test together, the slope is 1.75 $[1.63, 1.93]$.

The numbers show how far the benchmark budgets were from what this start needs. At about 1,000 features, it needed 154,000 to 214,000 fits before it held a subset with ten features, while the benchmark budgets were 2,000 to 12,000. On the line fitted to the test, a dataset with 5,000 features, such as gisette, would need about 2.6 million fits, but this lies beyond the tested range. The test cannot separate the number of features from the number of training rows. The three datasets with the fewest features also have the most rows (8,346 to 30,002), and the ratio to the prediction rises with the rows (Spearman correlation 0.77) as it falls with d (-0.89). Because each run stops when it reaches the

limit, the test also says nothing about the scores of the opposition-based start once it is there.

The robustness of the main results to runs that did not finish and to the choice of classifier is shown in Supplementary Section S3.

6. Discussion

At a fixed budget, AUF with a small limit measured mainly how far each start was from the small subset sizes, not how well the search worked among them. In every group of runs, the split of the AUF difference between the two starts was dominated by reach, mostly at limits that the OBL start never reached (Section 5.1). At the sizes both starts reached, the mean difference in test F1 was small under the decision tree and LightGBM and inconclusive under the random forest. Single datasets differed more: by up to $+0.036$ in favor of the sparse start (Adiac, LightGBM) and by -0.106 and -0.138 in favor of the OBL start on the two datasets with 8 and 11 test rows (BeetleFly and Coffee, random forest). A large AUF lead at a limit of ten features suggests that the leading method finds better small subsets; in the tested configurations, it meant mainly that the other method returned no small subset, or returned one in fewer runs. The same check applies to any comparison in which one method starts from, or stays near, large subsets, including comparisons of other feature-selection methods against an opposition-based NSGA-II: before a lead at a small limit is attributed to better search, it should be checked whether both methods reached that limit.

The analysis of Section 3 and the runs give a consistent account of why this happens. Beyond a few dozen features, an OBL initial population almost never contains a subset with ten or fewer features (Section 3.4), and the search moves down from about half of the features slowly: in the pilot, after 0.54 to 7.10 times the benchmark budget. The confirmatory test gives this a scale. On seven datasets not used before, the fits needed grew as about $d^{1.7}$, within a factor of two of the prediction from the pilot (Section 5.8). The line fitted to the test passes 12,000 fits, the largest benchmark budget, at about 210 features, close to the 163 to 226 features at which half of the benchmark runs reached the limit, and at 512 features it lies at about 55,000 fits. This agreement is an observation after the fact: the benchmark runs used three classifiers, budgets set per dataset, and mostly time series. The test measured the growth but not its cause; we do not know why the exponent is close to 1.7. Once small subsets are present, the large ones disappear under the tested variation and survival rules, whatever the start (Table 5), and balanced mutation leaves about $e^{-2k/d}$ of the children unchanged, so repeated subsets dominate the proposals without counting as fits. We did not vary crossover or selection, so we do not claim that this is the only possible explanation.

The sparse start is not a better feature-selection method; it searches in a different region. Its best test F1 at any size was lower by 0.007 to 0.029 on average, and its fronts ended by 36 features (Section 5.4). Neither tested start covered both ends of the front, and the uniform and log-uniform starts of the pilot lost their large subsets in the same way. Because the large subsets are lost during the search rather than missing at the start, keeping both ends probably requires a change in which subsets survive, for example letting the archive of the best subset of each size, which our runs keep only for reporting, take part

in selection, niching by subset size, or survival by size group [20]. We did not test such rules.

The improvement phase of our NSGAII-MIIP implementation did not compensate for the dense start: it raised reach by only 0.04 to 0.08, and the sparse start remained ahead of it by 0.46 to 0.64 (Section 5.5). In these runs and within the tested budgets, the start mattered far more than the improvement phase for whether the front reached small sizes.

For small subsets, a budget of fits is also a poor bound on running time. Without the stopping rule, single sparse runs lasted up to 563,390 generations in the designed experiment and 465,914 in the benchmark with the decision tree. On the four pilot datasets, the forced flip made sparse runs about 3 to 17 times faster with a similar AUF (Section 5.7); there, it lowered the cost rather than improving the search. A budget of fits is a fair basis for comparing how much evaluation two methods used, but it does not measure computing time; reporting both follows the distinction between evaluation budgets and computing time in optimization benchmarking [3].

These results lead to a short list of quantities to report alongside AUF in a comparison at a feature limit:

- whether each run reaches the limit, and after how much of the budget;
- the score at sizes that both methods reach, per dataset and as a mean with its interval;
- the best score at any size, which shows what a method gives up among large subsets;
- the numbers of proposed subsets and new model fits, and the elapsed time;
- which runs did not finish, and how they were treated in the analysis.

The budget should also be chosen with the number of features in mind. For the OBL start with these operators and the decision tree, reaching ten features took about 160,000 fits at a thousand features, and the number grew faster than in proportion to d. A budget well below this cannot show how the start compares among small subsets, and the comparison should then be described as one of reach at that budget.

The main reading persisted under the three tested treatments of missing runs, which are scenarios rather than bounds, and under all three classifiers (Supplementary Section S3).

The evidence has several limits. Most datasets come from a benchmark whose runs we analyzed after their results were known, and the pilot used datasets whose earlier results we had seen, so both guide the interpretation rather than confirm it. Only the relation between the budget and the number of features was tested on unseen data, and only for the OBL start under the decision tree, on 128 to 1,024 features; in that test, the number of features and the number of training rows varied together, and the measured exponent was lower than the predicted one. Thirty-eight of the 46 datasets are time series from the UCR and UEA archives, in which neighboring features are strongly correlated; none of the seven datasets of the confirmatory test is. Every dataset has a single split, so the five seeds measure the variability of the search, not of the data. We tested one family of operators, our implementation of those of NSGAII-MIIP, with a population of 100, and we did not compare its outputs with those of the authors' code. Exact library versions were not stored with each run, and the code commits of the random-forest runs were reconstructed rather than recorded. The benchmark budgets were calibrated on FASTENER and grow with the number of features, so the benchmark alone cannot separate the two effects. Only the pilot and the confirmatory test varied the budget; the pilot's three budgets are points on the same search trajectories, not independent runs, and the confirmatory runs stopped at

the limit, so they say nothing about quality after reaching it. Quality at matched sizes is known only for the few datasets on which both starts reach small sizes, and the rule that sets aside the smallest test sets was chosen after a first analysis. The pilot and the confirmatory test used a single classifier, the decision tree. Finally, the random split of EOData makes its test scores optimistic.

7. Conclusion

We asked whether differences at a small feature limit reflect search quality among small subsets or only whether the searches reach such subsets within a finite budget, and compared an opposition-based start, a sparse start, and our implementation of NSGAII-MIIP on up to 46 datasets (NSGAII-MIIP on 39), with a pilot on four and a confirmatory test on seven new datasets. At a limit of ten features, the split of the AUF difference between the two starts was dominated by reach, which made up 98 to 100 percent of the combined absolute reach and quality parts. At sizes both starts reached, the mean test F1 difference was small under two classifiers and inconclusive under the third. Reach depended on the number of features and the budget: the fits the OBL start needed to reach ten features grew as about $d^{1.7}$, a relation predicted from the pilot and confirmed on the new datasets. Under the tested variation and survival rules no start kept both ends of the front. A large AUF difference at a small limit is therefore not by itself evidence of better search, and it should be reported together with reach and the budget. Future work should test the matched-size reading on new splits and unseen datasets with the margin and the eligibility rule fixed in advance, look for a selection or archive rule that keeps both ends of the front, separate the number of features from the number of training rows in the budget relation and test it under other classifiers, and extend the comparison to wider datasets, larger budgets, and other operator families.

Acknowledgments. This work was supported by the Slovenian Research and Innovation Agency (ARIS) through the Gravitacija project Artificial Intelligence for Science (GC-0001), and by the European Union's Horizon Europe research and innovation programme under grant agreement No. 101189771 (DataPACT).

We acknowledge the Slovenian National Supercomputing Network (SLING) for access to computing infrastructure. Computations were performed on the Vega supercomputer at the Institute of Information Science (IZUM) in Maribor, Slovenia. We thank the HPC RIVR consortium and the EuroHPC Joint Undertaking for supporting this research through the provision of Vega computing resources.

We used generative AI tools (Claude by Anthropic; GPT-6.1 and Qwen2.5 for internal review) as assistants for writing and debugging code, running and logging experiments, computing tables and drawing figures from the experiment records, searching and summarizing related work, drafting and editing text, and internal review of the manuscript. Some design details of the experiments were proposed by these tools and adopted after the author's review. The research questions, hypotheses, decisions, and interpretation of the results are the author's. All output was checked by the author, who takes full responsibility for the content.

References

1. Auger, A., Bader, J., Brockhoff, D., Zitzler, E.: Hypervolume-based multiobjective optimization: Theoretical foundations and practical implications. Theoretical Computer Science 425, 75–103 (2012), https://doi.org/10.1016/j.tcs.2011.03.012
2. Bagnall, A., Dau, H.A., Lines, J., Flynn, M., Large, J., Bostrom, A., Southam, P., Keogh, E.: The UEA multivariate time series classification archive, 2018. arXiv:1811.00075 (2018), https://arxiv.org/abs/1811.00075
3. Bartz-Beielstein, T., Doerr, C., van den Berg, D., Bossek, J., Chandrasekaran, S., Eftimov, T., Fischbach, A., Kerschke, P., La Cava, W., López-Ibáñez, M., Malan, K.M., Moore, J.H., Naujoks, B., Orzechowski, P., Volz, V., Wagner, M., Weise, T.: Benchmarking in Optimization: Best Practice and Open Issues. arXiv preprint arXiv:2007.03488 (2020), https://doi.org/10.48550/arXiv.2007.03488
4. Cattelani, L., Fortino, V.: Improved NSGA-II algorithms for multi-objective biomarker discovery. Bioinformatics 38(Supplement_2), ii20–ii26 (2022), https://doi.org/10.1093/bioinformatics/btac463
5. Chu, S.C., Zhuang, Z., Pan, J.S., Mohamed, A.W., Hu, C.C.: Enhanced SparseEA for large-scale multi-objective feature selection problems. Complex & Intelligent Systems 10(1), 485–507 (2024), https://doi.org/10.1007/s40747-023-01177-2
6. Cong, Y., Wu, T., Yang, X., Zhu, Y., Tang, M., Yang, H., Liu, P.: Nested Evaluation of Sparse Multi-Objective Evolutionary Gene-Panel Discovery for Diabetic Nephropathy. Bioengineering 13(10), 1229 (2026), https://doi.org/10.3390/bioengineering13101229
7. Dau, H.A., Bagnall, A., Kamgar, K., Yeh, C.C.M., Zhu, Y., Gharghabi, S., Ratanamahatana, C.A., Keogh, E.: The UCR time series archive. IEEE/CAA Journal of Automatica Sinica 6(6), 1293–1305 (2019), https://doi.org/10.1109/JAS.2019.1911747
8. Deb, K., Pratap, A., Agarwal, S., Meyarivan, T.: A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Transactions on Evolutionary Computation 6(2), 182–197 (2002), https://doi.org/10.1109/4235.996017
9. Deniz, A., Kiziloz, H.E.: On initial population generation in feature subset selection. Expert Systems with Applications 137, 11–21 (2019), https://doi.org/10.1016/j.eswa.2019.06.063
10. Deniz, A., Kiziloz, H.E.: Boosting Initial Population in Multiobjective Feature Selection with Knowledge-Based Partitioning. In: 2022 International Joint Conference on Neural Networks (IJCNN). pp. 1–8 (2022), https://doi.org/10.1109/IJCNN55064.2022.9892123
11. Holm, S.: A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics 6(2), 65–70 (1979), https://www.jstor.org/stable/4615733
12. Jiao, R., Nguyen, B.H., Xue, B., Zhang, M.: A Survey on Evolutionary Multiobjective Feature Selection in Classification: Approaches, Applications, and Challenges. IEEE Transactions on Evolutionary Computation 28(4), 1156–1176 (2024), https://doi.org/10.1109/TEVC.2023.3292527
13. Jiao, R., Xue, B., Zhang, M.: Solving Multiobjective Feature Selection Problems in Classification via Problem Reformulation and Duplication Handling. IEEE Transactions on Evolutionary Computation 28(4), 846–860 (2024), https://doi.org/10.1109/TEVC.2022.3215745
14. Kelly, M., Longjohn, R., Nottingham, K.: The UCI Machine Learning Repository. University of California, Irvine (2023), https://archive.ics.uci.edu
15. Koprivec, F., Kenda, K., Šircelj, B.: FASTENER Feature Selection for Inference from Earth Observation Data. Entropy 22(11), 1198 (2020), https://doi.org/10.3390/e22111198

16. Kropp, I., Nejadhashemi, A.P., Deb, K.: Benefits of sparse population sampling in multi-objective evolutionary computing for large-Scale sparse optimization problems. Swarm and Evolutionary Computation 69, 101025 (2022), https://doi.org/10.1016/j.swevo.2021.101025
17. Li, A.D., He, Z., Wang, Q., Zhang, Y., Ma, Y.: A multi-objective evolutionary algorithm with mutual-information-guided improvement phase for feature selection in complex manufacturing processes. European Journal of Operational Research 323(3), 952–965 (2025), https://doi.org/10.1016/j.ejor.2024.12.036
18. Li, A.D., Xue, B., Zhang, M.: Multi-objective feature selection using hybridization of a genetic algorithm and direct multisearch for key quality characteristic selection. Information Sciences 523, 245–265 (2020), https://doi.org/10.1016/j.ins.2020.03.032
19. Li, J., Cheng, K., Wang, S., Morstatter, F., Trevino, R.P., Tang, J., Liu, H.: Feature Selection: A Data Perspective. ACM Computing Surveys 50(6), 1–45 (2017), https://doi.org/10.1145/3136625
20. Li, W., Jia, H., Han, C.: CORAL: A Rank-Memory Search Framework for Multi-Objective Feature Selection. Information 17(6), 593 (2026), https://doi.org/10.3390/info17060593
21. Oh, I.S., Lee, J.S., Moon, B.R.: Hybrid genetic algorithms for feature selection. IEEE Transactions on Pattern Analysis and Machine Intelligence 26(11), 1424–1437 (2004), https://doi.org/10.1109/TPAMI.2004.105
22. Šircelj, B., Kenda, K., Koprivec, F.: Land Patch Samples. PANGAEA (2020), https://doi.org/10.1594/PANGAEA.914271
23. Tian, Y., Zhang, X., Wang, C., Jin, Y.: An Evolutionary Algorithm for Large-Scale Sparse Multiobjective Optimization Problems. IEEE Transactions on Evolutionary Computation 24(2), 380–393 (2020), https://doi.org/10.1109/TEVC.2019.2918140
24. Tizhoosh, H.R.: Opposition-based learning: A new scheme for machine intelligence. In: International Conference on Computational Intelligence for Modelling, Control and Automation and International Conference on Intelligent Agents, Web Technologies and Internet Commerce (CIMCA-IAWTIC'06). vol. 1, pp. 695–701. IEEE (2005), https://doi.org/10.1109/CIMCA.2005.1631345
25. Vanschoren, J., van Rijn, J.N., Bischl, B., Torgo, L.: OpenML: networked science in machine learning. ACM SIGKDD Explorations Newsletter 15(2), 49–60 (2014), https://doi.org/10.1145/2641190.2641198
26. Wang, P., Xue, B., Liang, J., Zhang, M.: Differential Evolution With Duplication Analysis for Feature Selection in Classification. IEEE Transactions on Cybernetics 53(10), 6676–6689 (2023), https://doi.org/10.1109/TCYB.2022.3213236
27. Wang, Y., Fan, R., Cheng, L., Gong, B., Liu, J.: An Improved Adaptive NSGA-II with Multiple Filtering for High-Dimensional Feature Selection. Electronics 15(1), 236 (2026), https://doi.org/10.3390/electronics15010236
28. Xue, B., Zhang, M., Browne, W.N., Yao, X.: A Survey on Evolutionary Computation Approaches to Feature Selection. IEEE Transactions on Evolutionary Computation 20(4), 606–626 (2016), https://doi.org/10.1109/TEVC.2015.2504420
29. Xue, Y., Cai, X., Neri, F.: A multi-objective evolutionary algorithm with interval based initialization and self-adaptive crossover operator for large-scale feature selection in classification. Applied Soft Computing 127, 109420 (2022), https://doi.org/10.1016/j.asoc.2022.109420
30. Zanjani Miyandoab, S., Rahnamayan, S., Asilian Bidgoli, A., Ebrahimi, S., Makrehchi, M.: Enhancing Diversity in Multi-Objective Feature Selection. In: 2024 IEEE Congress on Evolutionary Computation (CEC). pp. 1–8 (2024), https://doi.org/10.1109/CEC60901.2024.10612084

31. Zanjani Miyandoab, S., Rahnamayan, S., Asilian Bidgoli, A., Makrehchi, M.: A comparative study on population-based multi-objective feature selection. International Journal of Machine Learning and Cybernetics 17(3), 105 (2026), https://doi.org/10.1007/s13042-025-02921-5